

Lab 5: Asynchronous Operations
Advanced Computerarchitecture

(B-KUL-JLI1QS)

Reinout Annaert, Michiel Van Nieuwenhove

March 24, 2024



Session Date:	18/03/2024
Class:	MElsw
Instructor:	Alex Marinšek

In this lab session, we will delve into the realm of asynchronous operations in CUDA, focusing on optimizing the processing of four independent arrays using sum, product, minimum, and maximum operations using reduction. We will explore two different approaches to achieve this task.

In the first approach, we will perform all operations sequentially, synchronously executing each task one after the other. This synchronous approach allows us to process the arrays in a straightforward manner but may not fully utilize the computational power of the GPU. In the second approach, we will adopt a more advanced strategy. Here, we will introduce asynchronous operations to the mix. While one set of data is being processed asynchronously on the GPU, we will simultaneously generate the next array on the CPU. By overlapping the data generation and processing tasks, we aim to maximize GPU utilization.

To acccomplish the synchronous method, we implemented the operations as we usually would. First, generating the array. Then using 'cudaMalloc' and 'cudaMemcpy' to load the data onto the GPU. After this, the operation can run. We do this sequentially for every operation. In contrast, the asynchronous approach follows a similar pattern, but not in a strictly sequential manner. While the data of one operation is being processed on the GPU, the array for the next operation can already be created. This concurrent operation enables the asynchronous function to operate more efficiently.

The execution times presented in Figure 1 are averages of 100 measurements for each array size. The asynchronous and synchronous approaches were both measured under the same conditions to provide a fair comparison.

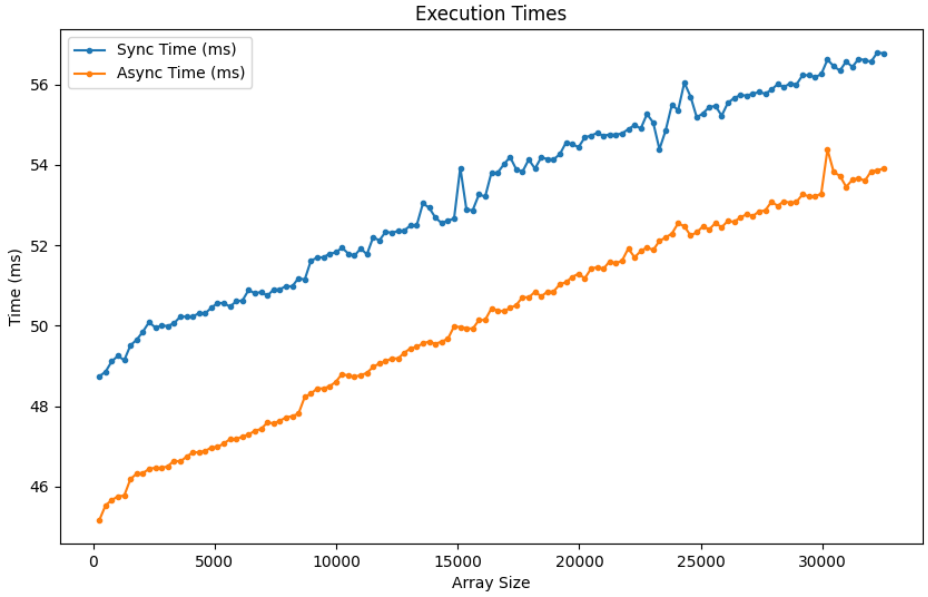


Figure 1: Execution Times in function of Array Size

After analyzing the execution times for both asynchronous and synchronous approaches, we can conclude that the asynchronous approach consistently outperforms the synchronous one. This is clearly depicted in the graph, where the asynchronous execution times are consistently lower than their synchronous counterparts. It seems that there is a more or less constant difference in execution time over the different array sizes. We implemented striding for larger array sizes, so we see that for higher array sizes the total execution time rises. This is because the threads have to execute more tasks when the amount of data increases.



This finding suggests that employing asynchronous operations is an effective strategy for reducing overall processing time and improving the efficiency of GPU resource utilization. By executing tasks in parallel and maximizing the overlap between data generation and processing, the asynchronous approach can offer significant benefits over the more traditional synchronous approach.

The GitHub repository containing the source code can be found at *Github Repository*